TO_CAD Pipeline Documentation

Converting Mesh-Based TO Geometries to
Parametric CAD Models

James Jacouris

Version 1.0

April 23, 2026

Contents

List of Figures **5**

List of Tables **6**

1 Introduction **7**

2 Getting Started **8**

2.1 Installation . . . 8

2.1.1 Python Dependencies . . . 8

2.1.2 Verifying the Installation . . . 8

2.1.3 FreeCAD Macro Setup . . . 9

2.2 Quick Start . . . 9

2.2.1 Typical Output Files . . . 9

2.3 Pipeline Walkthrough . . . 10

2.3.1 Stage 0 — Topology Optimisation (Top3D) . . . 10

2.3.2 Stage 1 — Skeleton Reconstruction . . . 10

2.3.3 Stage 2 — Size Optimisation . . . 11

2.3.4 Stage 3 — Layout Optimisation . . . 11

2.3.5 Hybrid Beam+Plate Mode . . . 11

2.3.6 Curved Beam Mode . . . 12

2.3.7 External Mesh Input . . . 12

2.3.8 Symmetry Enforcement . . . 12

2.4 FreeCAD Export . . . 12

2.4.1 Geometry Created . . . 12

2.4.2 Running the Macro . . . 13

2.4.3 Supported JSON Fields . . . 13

2.4.4 Known Limitations . . . 13

3 Architecture **14**

3.1 Architecture Overview . . . 14

3.1.1 Repository Layout . . . 14

3.1.2 Module Dependency Graph . . . 15

3.1.3 Design Decisions . . . 15
3.2 Data Flow . . . 16
3.2.1 Stage 0 → Stage 1: NPZ Format . . . 16
3.2.2 Stage 1 Output: JSON Schema . . . 16
3.2.3 Stages 2 & 3: In-Memory Arrays . . . 17
3.2.4 Pipeline History Object . . . 17
3.2.5 External Mesh Input: NPZ Format . . . 17
3.2.6 Symmetry Data Flow . . . 17
3.2.7 BC Tag Propagation . . . 17
3.3 Algorithms . . . 18
3.3.1 Top3D — Topology Optimisation (Stage 0) . . . 18
3.3.2 Skeletonisation (Stage 1) . . . 19
3.3.3 Two-Signal Zone Classification (Hybrid) . . . 20
3.3.4 Hybrid Surface Extraction and Joint Creation . . . 21
3.3.5 Frame FEA . . . 21
3.3.6 Frame Re-optimisation (Stages 2–3) . . . 22
3.3.7 External Mesh Input . . . 23
3.3.8 Symmetry Enforcement . . . 23
3.4 JSON Output Schema . . . 23
3.4.1 Top-Level Structure . . . 23
3.4.2 `metadata` . . . 23
3.4.3 `graph` . . . 24
3.4.4 `curves` . . . 24
3.4.5 `plates` . . . 24
3.4.6 `joints` . . . 24
3.4.7 `history` . . . 25

4 CLI Reference **27**
4.1 `run_pipeline.py` — Main Pipeline . . . 27
4.1.1 Synopsis . . . 27
4.1.2 Examples . . . 27
4.1.3 Argument Reference . . . 28
4.2 `run_top3d.py` — Topology Optimisation . . . 30
4.2.1 Synopsis and Examples . . . 30
4.2.2 Argument Reference . . . 30
4.2.3 Problem Types . . . 30
4.3 `tune_parameters.py` — Parameter Tuning . . . 30
4.3.1 Synopsis and Examples . . . 30
4.3.2 Argument Reference . . . 32
4.3.3 Search Space . . . 32
4.3.4 Objective Function . . . 32

5 API Reference **33**
5.1 Pipelines — `src.pipelines.baseline_yin` . . . 33
5.1.1 `reconstruct` . . . 33
5.1.2 `thinning` . . . 34
5.1.3 `topology` . . . 34
5.1.4 `graph` . . . 34
5.1.5 `postprocessing` . . . 35
5.1.6 `plate_extraction` . . . 35
5.1.7 `joint_creation` . . . 36
5.1.8 `surface_fitting` . . . 36
5.1.9 `visualization` . . . 36
5.1.10 `mesh_voxelizer` . . . 36
5.2 Optimization — `src.optimization` . . . 36
5.2.1 `top3d` . . . 36
5.2.2 `fem` . . . 37
5.2.3 `size_opt` . . . 38
5.2.4 `layout_opt` . . . 38
5.2.5 `symmetry` . . . 39
5.3 Problems — `src.problems` . . . 39
5.3.1 Factory: `load_problem_config` . . . 39
5.3.2 `TaggedProblem` . . . 39
5.3.3 `CantileverSetup` . . . 39
5.3.4 `GenericProblem` . . . 39
5.4 Curves — `src.curves.spline` . . . 39
5.5 Export — `src.export.freecad_reconstruct` . . . 40
5.6 Tuning — `src.tuning` . . . 40
5.6.1 `pipeline_runner` . . . 40
5.6.2 `metrics` . . . 40

6 Developer Guide **41**
6.1 Contributing . . . 41
6.1.1 Branch Policy . . . 41
6.1.2 Code Style . . . 41
6.1.3 Adding a New Problem . . . 41
6.1.4 Adding a New Thinning Mode . . . 42
6.1.5 Building the Docs . . . 42
6.2 Testing and Validation . . . 42
6.2.1 Smoke Test — Import Check . . . 42
6.2.2 Beam-Only Regression . . . 42
6.2.3 Hybrid Regression . . . 42
6.2.4 Parameter Tuning Validation . . . 43

6.2.5 Docs Build Check 43

List of Figures

3.1 Pipeline module dependencies. Dashed edges indicate hybrid-only or optional paths. Green-filled modules are hybrid-only; orange-filled are optional. 15
3.2 Roof structure problem setup ($40 \times 40 \times 20$ elements). 25
3.3 Frame-supported beam problem setup. 26

List of Tables

1.1 Pipeline stages. 7

2.1 Key Stage 1 parameters. 11
2.2 JSON fields consumed by the FreeCAD macro. 13

3.1 Thinning modes and preserved topology. 20
3.2 Metadata fields. 23
3.3 Graph fields. 24

4.1 Complete argument reference for `run_pipeline.py`. 28
4.2 Arguments for `run_top3d.py`. 31
4.3 Available problem types. 31
4.4 Arguments for `tune_parameters.py`. 32
4.5 Optuna search space. 32

Chapter 1

Introduction

TO_CAD converts mesh-based topology optimisation (TO) geometries into editable parametric CAD models (beams, plates, and fused solid bodies) ready for FreeCAD. The pipeline chains four stages:

Table 1.1: Pipeline stages.

Stage	Script / Module	What it does
0	`run_top3d.py`	3-D topology optimisation (Python Top3D) → density `.npz`
1	`reconstruct`	Yin medial-axis thinning → beam graph + plate regions → `.json`
2	`size_opt`	Optimality-Criteria cross-section sizing
3	`layout_opt`	L-BFGS-B node-position layout optimisation

The final JSON is imported into FreeCAD via `freecad_reconstruct.py`.

Additional capabilities include curved Bézier beams (`--curved`), external mesh input from STL/OBJ files (`--mesh_input`), mirror-half symmetry enforcement (`--symmetry`), and a fused solid body for STEP export.

Chapter 2

Getting Started

2.1 Installation

2.1.1 Python Dependencies

Python 3.9 or later is required. Create a virtual environment and install dependencies:

```
python -m venv .venv
source .venv/bin/activate

# Core
pip install numpy scipy numba matplotlib

# Visualisation (optional)
pip install open3d

# Parameter tuning (optional)
pip install optuna

# Documentation (optional)
pip install sphinx sphinx-rtd-theme
```

2.1.2 Verifying the Installation

```
python -c "
from src.optimization.top3d import Top3D
from src.pipelines.baseline_yin.reconstruct import reconstruct_npz
from src.optimization.size_opt import optimize_size
from src.optimization.layout_opt import optimize_layout
from src.problems import load_problem_config
print('All imports OK')
"
```

2.1.3 FreeCAD Macro Setup

Copy `src/export/freecad_reconstruct.py` to `~/.FreeCAD/Macro/` and run from the FreeCAD GUI via **Macro** → **Macros...**.

2.2 Quick Start

A four-command workflow demonstrates the full pipeline.

Step 1 — Topology Optimisation

```
python run_top3d.py \
    --nelx 60 --nely 20 --nelz 4 \
    --volfrac 0.3 --penal 3.0 --rmin 1.5 \
    --problem cantilever \
    --output output/hybrid_v2/cantilever_top3d.npz
```

Step 2 — Skeleton Reconstruction + Optimisation

```
python run_pipeline.py \
    --skip_top3d \
    --top3d_npz output/hybrid_v2/cantilever_top3d.npz \
    --opt_loops 2 \
    --output cantilever.json
```

Step 3 — Interactive Preview (optional)

```
python run_pipeline.py \
    --skip_top3d \
    --top3d_npz output/hybrid_v2/cantilever_top3d.npz \
    --visualize --output debug.json
```

Step 4 — FreeCAD Import

Open FreeCAD, run the macro, and select the `cantilever_history.json` file.

2.2.1 Typical Output Files

- `*_top3d.npz` — density field from Stage 0
- `*_1_reconstructed.json` — Stage 1 skeleton
- `*_3_sized_loop1.json` — after size optimisation
- `*_2_layout_loop1.json` — after layout optimisation
- `*.json` — final output
- `*_history.json` — full history for FreeCAD timeline

Tip: For a one-shot run (Stages 0–3 combined):

```
python run_pipeline.py --nelx 60 --nely 20 --nelz 4 \
```

```
    --volfrac 0.3 --opt_loops 2 --output cantilever.json
```

2.3 Pipeline Walkthrough

2.3.1 Stage 0 — Topology Optimisation (Top3D)

Entry point: `run_top3d.py` (called as subprocess from `run_pipeline.py` when `--skip_top3d` is *not* given).

Algorithm: Pure-Python port of Sigmund's 88-line MATLAB `top3d` code (`src.optimization.top3d`). Uses a SIMP material model with OC density updates and a mesh-independence filter.

Boundary conditions: Set via `--problem`:

- `cantilever` — fixed at $x = 0$, point load at $x = n_x$
- `roof` — four corner supports, central downward load
- `bridge` — bottom surface fixed, distributed top load
- `tagged` (pipeline default) — BC tags propagated from `run_top3d.py` into the `bc_tags` array stored in the NPZ

Key outputs (stored in `.npz`):

```
npz['rho']      # shape (nely, nelx, nelz), float64, density in [0, 1]
npz['bc_tags'] # shape (nely, nelx, nelz), int32; 1=fixed, 2=loaded, 3=passive
```

2.3.2 Stage 1 — Skeleton Reconstruction

Entry point: `reconstruct_npz()` in `src.pipelines.baseline_yin.reconstruct`.

Sub-steps:

1. **Threshold** — `solid = rho > vol_thresh` creates a binary voxel mask.
2. **EDT** — `scipy.ndimage.distance_transform_edt(solid)` pre-computes the Euclidean distance transform.
3. **Thinning** — `thin_grid_yin()` iteratively removes simple points until a medial axis remains. Mode selects topology: `mode=0` (curve-preserving), `mode=3` (hybrid).
4. **Graph extraction** — classifies skeleton voxels into endpoints (≤ 1 neighbour), body (exactly 2), and junctions (≥ 3), then traces edges between non-body voxels.
5. **Laplacian smoothing** — three iterations of Laplacian averaging on edge waypoints to remove staircase artefacts.
6. **Post-processing** — collapse short edges, prune branch tips, RDP simplification, compute per-edge radii.
7. **Initial radius** — uniform radius: $r_0 = \sqrt{V_{\text{solid}}/(\pi \sum_i L_i)}$.
8. **[Hybrid only]** — two-signal zone classification, plate extraction, beam-plate joint creation.

9. **Export** — writes the graph + plate data to `.json`.

Table 2.1: Key Stage 1 parameters.

Parameter	Default	Effect
`--vol_thresh`	0.3	Density threshold; lower → more voxels survive
`--prune_len`	2.0 mm	Remove branch tips shorter than this
`--collapse_thresh`	2.0 mm	Merge nodes connected by edges shorter than this
`--rdp`	1.0 mm	Ramer–Douglas–Peucker epsilon for polyline simplification
`--radius_mode`	`uniform`	`uniform` = volume-matching; `edt` = geometric from EDT

2.3.3 Stage 2 — Size Optimisation

Entry point: `optimize_size()` in `src.optimization.size_opt`.

Uses the Optimality Criteria (OC) method to resize beam cross-section radii so that the total frame volume matches `target_volume` while minimising compliance (maximising stiffness). Iterates until the relative change in radii is below 10^{-3} or `--iters` is reached.

2.3.4 Stage 3 — Layout Optimisation

Entry point: `optimize_layout()` in `src.optimization.layout_opt`.

Uses `scipy.optimize.minimize` with L-BFGS-B to move node positions along compliance gradients, subject to:

- A move limit (`--limit`, default 5 mm per iteration)
- Design-domain box constraints
- Node snapping (`--snap`) to merge nodes that drift within tolerance

Stages 2 and 3 are interleaved for `--opt_loops` iterations. Each loop runs *Size first, then Layout.*

2.3.5 Hybrid Beam+Plate Mode

Enabled with `--hybrid`.

After thinning (`mode=3`), the skeleton is classified into beam and plate zones using a two-signal approach:

- **Signal A** — skeleton set-difference: voxels present in the `mode=3` skeleton but absent from a `mode=0` skeleton are plate candidates.
- **Signal B** — octant plane pattern count ≥ 4 with ≥ 3 foreground neighbours identifies surface-like voxels directly.

Combined candidates are filtered by connected-component size and global linearity to reject noise. Final classification:

- **Plate voxels** (`zone == 1`): surface-like topology

- **Beam voxels** (`zone == 2`): curve-like topology

Plate regions are expanded back to full thickness via morphological dilation, smoothed with Taubin's λ/μ filter, and assigned EDT-based thickness ($2 \times$ EDT $\times$ pitch). Beam endpoints near a plate boundary are snapped to a shared node by KD-tree lookup.

2.3.6 Curved Beam Mode

`--curved` fits cubic Bézier curves through skeleton waypoints. When enabled, the FEM assembler can use IGA Timoshenko curved-beam elements (cubic Bernstein basis, 24-DOF, statically condensed to 12-DOF) for beams with significant curvature, while short or straight beams continue to use standard Euler–Bernoulli elements. After each optimisation stage, control points are re-fitted from updated node positions.

2.3.7 External Mesh Input

`--mesh_input <path>` allows the pipeline to accept an external STL or OBJ mesh instead of running Top3D. The mesh is voxelised at the pitch specified by `--mesh_pitch` using trimesh.

2.3.8 Symmetry Enforcement

`--symmetry x|y|z` enforces mirror symmetry about the specified axis. The skeleton is split at the symmetry plane, one half is retained for optimisation, and the result is reflected back. Nodes on the symmetry plane are fixed in the normal direction during layout optimisation.

2.4 FreeCAD Export

The FreeCAD macro (`src.export.freecad_reconstruct`) reads the pipeline `.json` output and creates 3-D CAD geometry inside a FreeCAD document.

2.4.1 Geometry Created

- **Beams** — swept circular tubes with hemispherical end caps, coloured with a radius heatmap (blue $\rightarrow$ red).
- **Curved beams** — `Part.BezierCurve` piped through the cross-section circle.
- **Plates** — polygon faces or B-spline surfaces; curved plates use offset mid-surfaces along vertex normals.
- **Fused body** — a single solid created by batched `multiFuse` and `removeSplitter()`, suitable for STEP export.
- **History timeline** — one FreeCAD group per pipeline stage for before/after comparison.
- **Density legend** — 10 coloured reference cubes (red $\rightarrow$ green) for the initial-voxels stage.

2.4.2 Running the Macro

1. Open FreeCAD (0.21+).
2. **Macro** → **Macros...** → Browse to `freecad_reconstruct.py` → **Execute**.
3. Select the `<name>_history.json` file in the file picker.
4. Wait for geometry creation (30–60 s for large models).

2.4.3 Supported JSON Fields

Table 2.2: JSON fields consumed by the FreeCAD macro.

Field	Description
`metadata.pitch`	Voxel size in mm; sets the global scale
`graph.nodes`	Node positions `[[x, y, z], ...]`
`graph.edges`	Edge list `[[u, v, r], ...]`
`curves`	Per-edge visualisation data (straight or Bézier)
`plates`	Plate geometry (vertices, connectivity, optional B-spline)
`history`	Voxel snapshots at each processing stage
`stages`	Graph states at each optimisation loop

2.4.4 Known Limitations

- Mid-surface plate geometry requires Open3D, which is not available inside the FreeCAD Python interpreter.
- Fused body Boolean operations can be slow for > 2000 beams.
- Very large models (> 5000 beams) may exceed FreeCAD's undo buffer.

Chapter 3

Architecture

3.1 Architecture Overview

3.1.1 Repository Layout

```
TO_CAD/
+-- run_pipeline.py            # Main orchestrator (CLI + Python API)
+-- run_top3d.py               # Stage 0: topology optimisation
+-- run_top3d_rocker_arm.py    # Rocker-arm specific Top3D script
+-- tune_parameters.py         # Optuna parameter search
|
+-- src/
|   +-- optimization/
|   |   +-- top3d.py           # Python Top3D solver (SIMP + OC)
|   |   +-- fem.py             # 3-D frame FEA (straight + curved)
|   |   +-- size_opt.py        # OC radius sizing
|   |   +-- layout_opt.py      # L-BFGS-B node-position optimisation
|   |   +-- symmetry.py        # Mirror-half symmetry enforcement
|   |
|   +-- pipelines/
|   |   +-- baseline_yin/
|   |       +-- reconstruct.py      # Stage 1 orchestrator + JSON export
|   |       +-- thinning.py         # Yin medial-axis thinning
|   |       +-- topology.py         # Topological predicates (Numba)
|   |       +-- graph.py            # Skeleton -> graph extraction
|   |       +-- postprocessing.py   # Graph pruning / cleanup
|   |       +-- plate_extraction.py # Plate region recovery (hybrid)
|   |       +-- joint_creation.py   # Beam-plate joint snapping
|   |       +-- surface_fitting.py  # B-spline plate mid-surfaces
|   |       +-- visualization.py    # Open3D / Matplotlib visualisation
|   |
|   +-- mesh_import/
|   |   +-- mesh_voxelizer.py  # External STL/OBJ -> voxel grid
|   |
|   +-- problems/
```

```
|   |   +-- __init__.py        # load_problem_config() factory
|   |   +-- cantilever.py
|   |   +-- tagged_problem.py  # Tag-based BC propagation
|   |   +-- generic.py
|   |
|   +-- curves/
|   |   +-- spline.py          # Cubic Bezier fitting & sanitisation
|   |
|   +-- export/
|   |   +-- freecad_reconstruct.py  # FreeCAD macro
|   |
|   +-- tuning/
|       +-- pipeline_runner.py  # Subprocess wrapper for trials
|       +-- metrics.py          # Geometry fidelity metrics
|
+-- docs/                       # Documentation
```

3.1.2 Module Dependency Graph

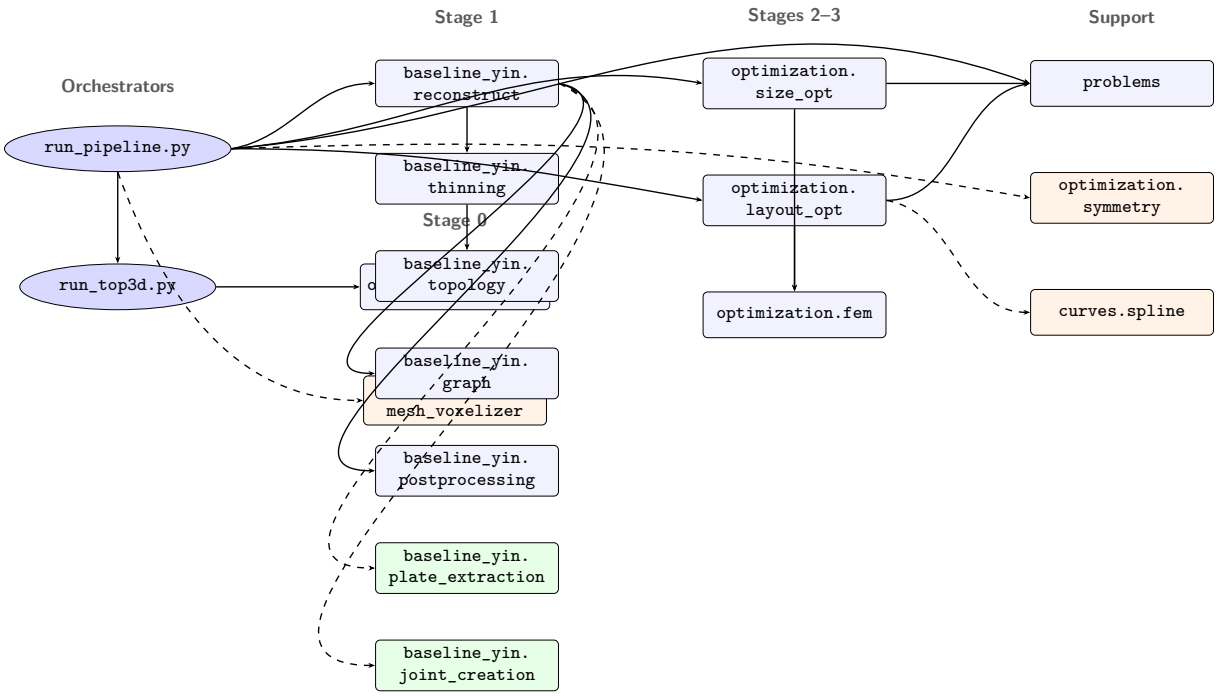

Figure 3.1: Pipeline module dependencies. Dashed edges indicate hybrid-only or optional paths. Green-filled modules are hybrid-only; orange-filled are optional.

3.1.3 Design Decisions

Classify after thinning (hybrid mode)

Zone classification is applied *after* a single hybrid thinning pass (`mode=3`), not to the original solid. A two-signal approach combines skeleton set-difference with octant plane

pattern counting.

Separate JSON edge format `[u, v, r]`

All JSON edge lists use exactly three elements: node index `u`, node index `v`, radius `r` (mm).

Direct Python API for Stage 1

`reconstruct_npz()` is a clean Python-callable API. `run_pipeline.py` calls it directly (no subprocess).

Dual FEM element library

The default uses 12-DOF straight Euler–Bernoulli beam elements. When `--curved` is specified, an IGA Timoshenko element is available; interior DOFs are statically condensed to a 12-DOF interface.

Shared beam–plate volume budget

In hybrid mode, beam radii and plate thicknesses are co-optimised under a single volume constraint via the OC update.

Symmetry by mirror-half skeleton

When `--symmetry` is given, the skeleton is split at the symmetry plane, one half is optimised, and the result is reflected back.

3.2 Data Flow

3.2.1 Stage 0 → Stage 1: NPZ Format

`run_top3d.py` writes a NumPy `.npz` archive:

```
np.savez(output_path,
    rho=rho,         # shape (nely, nelx, nelz), float64 in [0, 1]
    bc_tags=bc_tags  # shape (nely, nelx, nelz), int32; 1=fixed, 2=loaded
)
```

Index order: Top3D uses `(nely, nelx, nelz)` (row-major MATLAB convention). When converting to world coordinates, indices are reordered to `[nelx, nely, nelz]` by taking `indices[:, [1, 0, 2]]`.

3.2.2 Stage 1 Output: JSON Schema

Key top-level keys (see Section 3.4 for the full reference):

```
{
  "metadata": { "pitch": 1.0, "units": "mm", "vol_thresh": 0.3, ... },
  "graph":    { "nodes": [...], "edges": [...], "node_tags": {...} },
  "curves":   [...],
  "plates":   [...],
  "joints":   [...],
```

```
  "history":  [...]
}
```

3.2.3 Stages 2 & 3: In-Memory Arrays

Size and layout optimisation operate on NumPy arrays extracted from the JSON:

```
nodes = np.array(data['graph']['nodes'])   # (N, 3) float64, mm
edges = np.array([[e[0], e[1]] for e in data['graph']['edges']], dtype=int)
radii = np.array([e[2] for e in data['graph']['edges']])  # (M,) mm
```

3.2.4 Pipeline History Object

`run_pipeline.py` assembles a history object for FreeCAD containing per-voxel snapshots (type `"voxels"`) and per-graph snapshots (type `"graph"`). The `stages` list contains the full geometry at each optimisation stage for the FreeCAD timeline feature.

3.2.5 External Mesh Input: NPZ Format

When `--mesh_input` is given, the NPZ contains only `rho` (binary, float32); no `bc_tags` are generated.

```
np.savez(output_path,
    rho=rho  # shape (nely, nelx, nelz), float32, binary {0.0, 1.0}
)
```

3.2.6 Symmetry Data Flow

When `--symmetry` is specified:

1. Edges crossing the symmetry plane are split at the plane intersection.
2. One half of the skeleton is retained.
3. After optimisation, the result is reflected and merged back.
4. Nodes on the symmetry plane are constrained in the plane-normal direction during layout optimisation.

3.2.7 BC Tag Propagation

BC tags flow from Top3D through the entire pipeline:

1. `run_top3d.py` sets `bc_tags[voxel] = 1` (fixed) or `2` (loaded).
2. `thin_grid_yin()` preserves tagged voxels from deletion.
3. `extract_graph()` assigns `node_tags` by majority vote of voxel tags at each node cluster.
4. `TaggedProblem` reads `node_tags` from the JSON and applies fixed DOFs / loads accordingly.

3.3 Algorithms

3.3.1 Top3D — Topology Optimisation (Stage 0)

`src.optimization.top3d` implements the 3-D extension of Sigmund's 88-line MATLAB `top` code, extended with BC tag generation, a conjugate-gradient solver, and convergence history tracking.

Material model: SIMP (Solid Isotropic Material with Penalisation)

$$E(\rho_e) = E_{\min} + \rho_e^p (E_0 - E_{\min}) \tag{3.1}$$

where p is the penalisation exponent (`--penal`, default 3), E_0 is the solid modulus, and $E_{\min} = 10^{-9}$.

Objective: Minimise compliance $C = \mathbf{F}^\top \mathbf{u}$ subject to a volume fraction limit V_f:

$$\min_{\boldsymbol{\rho}} \; C = \mathbf{F}^\top \mathbf{u} \quad \text{s.t.} \quad \mathbf{K}\mathbf{u} = \mathbf{F}, \quad \frac{1}{V_0} \sum_e \rho_e v_e \leq V_f, \quad 0 \leq \rho_e \leq 1 \tag{3.2}$$

Sensitivity:

$$\frac{\partial C}{\partial \rho_e} = -p \left(E_0 - E_{\min}\right) \rho_e^{p-1} \, \mathbf{u}_e^\top \mathbf{K}_E \mathbf{u}_e \tag{3.3}$$

where $\mathbf{K}_E$ is the unit-modulus element stiffness matrix. Sensitivities are averaged over each element's neighbourhood within radius $r_{\min}$ to suppress checkerboard patterns.

Density update: Optimality Criteria

$$\rho_e^{\text{new}} = \rho_e \cdot B_e^\eta \tag{3.4}$$

where $B_e = -\partial C / \partial \rho_e / (\lambda \, \partial V / \partial \rho_e)$ and λ is found by bisection to satisfy the volume constraint. Iteration continues until $\max_e |\Delta \rho_e| < 0.01$.

Mesh-independence filter: Weighted average over a ball of radius `--rmin`

$$\hat{\rho}_e = \frac{\sum_f H_{ef} \rho_f}{\sum_f H_{ef}}, \quad H_{ef} = \max(0, r_{\min} - \Delta(e, f)) \tag{3.5}$$

Element: 8-node hexahedral H8 with trilinear shape functions. Stiffness matrix pre-computed by `lk_H8()` and assembled via vectorised indexing. Sparse system solved with `scipy.sparse.linalg.spsolve`.

Binarisation: The converged density field is binarised before Stage 1:

$$\text{solid}_e = \begin{cases} 1 & \rho_e > v_t \\ 0 & \text{otherwise} \end{cases} \tag{3.6}$$

where $v_t = 0.30$ (`--vol_thresh`). BC-tagged voxels are retained regardless of density.

Boundary-Condition Tag Generation

Before optimisation, each element is assigned an integer tag encoding its structural role:

- `tag = 1` **(support)**: shares a node with a fixed-displacement DOF.
- `tag = 2` **(load)**: shares a node with an applied force.
- `tag = 3` **(passive)**: excluded from the design domain.

Tags 1 and 2 propagate through Stages 1–3, serving two roles: (1) thinning protection — tagged voxels are never deleted during skeletonisation; and (2) frame boundary conditions — tagged nodes supply fixed DOFs and loads to the frame solver.

3.3.2 Skeletonisation (Stage 1)

Yin Medial-Axis Thinning

`src.pipelines.baseline_yin.thinning` implements the 3-D parallel directional thinning algorithm by Yin (2002).

Core idea: Iteratively peel voxels from the surface of the binary solid, one layer at a time, until only a one-voxel-wide skeleton remains.

Simple point test: A voxel is only removed if its deletion does not disconnect any material or merge any void regions. The test examines the voxel's $3 \times 3 \times 3$ neighbourhood: the 26-connected material must form a single cluster, and the 6-connected void must also form a single cluster.

Directional sweeps: Each iteration sweeps in 6 axis-aligned directions ($\pm X, \pm Y, \pm Z$). Candidates are re-checked before deletion because an earlier removal can change a later candidate's neighbourhood.

Unconditionally excluded voxels:

1. **BC-tagged voxels** (`tag > 0`): support and load voxels are preserved.
2. **End voxels** (≤ 1 neighbour): prevents thin chains from eroding entirely.
3. **Surface points** (mode 1/3 only): voxels bounding a 2-D medial surface are retained.

Numba acceleration: Critical loops use `@njit(parallel=True)`.

Thinning modes:

Table 3.1: Thinning modes and preserved topology.

Mode	Name	Preserved topology
0	Curve-preserving	Curve endpoints; surfaces collapsed to medial curves
1	Surface-preserving	Surface voxels retained; curves collapsed
3	Hybrid (surface + curve)	Both endpoints and surface voxels preserved

Graph Extraction

`extract_graph()` converts skeleton voxels into a graph:

1. **BC tag cluster collapsing**: Connected clusters of BC-tagged voxels are collapsed to single centroid nodes.
2. **Node classification**: endpoint (≤ 1 neighbour), body (exactly 2), junction (≥ 3).
3. **Edge tracing**: BFS through degree-2 voxels traces the skeleton path between nodes.
4. **Laplacian smoothing**: Three iterations; interior waypoints move halfway toward midpoint of neighbours.
5. **BC tag assignment**: each node inherits the dominant BC tag.

Graph Post-processing

1. **Edge collapse**: Pairs connected by edges shorter than `--collapse_thresh` are merged.
2. **Branch pruning**: Degree-1 leaves on edges shorter than `--prune_len` are removed.
3. **Geometric simplification**: Ramer–Douglas–Peucker algorithm reduces waypoints within `--rdp` tolerance.

Initial Radius Assignment

Each edge receives an initial cross-sectional radius by uniform volume-matching:

$$r_0 = \sqrt{\frac{V_{\text{solid}}}{\pi \sum_i L_i}} \tag{3.7}$$

where V_{solid} is the total solid voxel volume and L_i the smoothed polyline length of edge i.

3.3.3 Two-Signal Zone Classification (Hybrid)

`classify_skeleton_post_thinning()` partitions the skeleton into surface and beam zones by combining two complementary topological signals.

Signal A (set-difference): Voxels present in the hybrid skeleton (`mode 3`) but absent from a curve-only skeleton (`mode 0`):

$$A = \mathcal{S}_3 \setminus \mathcal{S}_0 \tag{3.8}$$

Signal B (octant plane patterns): A voxel is a surface candidate if ≥ 4 octants match known planar configurations AND the voxel has ≥ 3 skeleton neighbours.

Combination: Plate candidates are the union:

$$\text{candidates} = A \cup B \tag{3.9}$$

The union is segmented into 26-connected regions, filtered by minimum size and elongation, and assigned as surface zones. Remaining voxels become beam zones.

3.3.4 Hybrid Surface Extraction and Joint Creation

Surface extraction: Full-thickness regions are recovered by morphological dilation into the original solid, constrained to avoid beam zones. The boundary is meshed, smoothed by Taubin filtering, and annotated with per-vertex thickness: $t_v = 2 \times \text{EDT}(v) \times \text{pitch}$.

Joint creation: Beam nodes near a surface boundary are snapped onto the nearest surface vertex using a KD-tree, creating shared nodes at each beam-to-surface interface.

3.3.5 Frame FEA

Straight Beam Element

`solve_frame()` assembles a 3-D Euler–Bernoulli frame stiffness matrix.

Element: 2-node 3-D beam with 6 DOF per node $(u_x, u_y, u_z, \theta_x, \theta_y, \theta_z)$. Local stiffness $\mathbf{K}_e$ (12×12) is computed analytically from beam length, Young's modulus E, and cross-section radius r (area $A = \pi r^2$, moment of inertia $I = \pi r^4/4$).

Assembly:

$$\mathbf{K} = \sum_e \mathbf{T}_e^\top \mathbf{K}_e \mathbf{T}_e \tag{3.10}$$

where $\mathbf{T}_e$ is the 12×12 rotation matrix aligning the local beam axis to the global frame.

Solve: Sparse LU via `spsolve(K_ff, f_f)` where subscript f denotes free DOFs.

Compliance: $C = \mathbf{F}^\top \mathbf{u}$ (scalar).

Gradient: $\partial C/\partial r_e = -\mathbf{u}_e^\top (\partial \mathbf{K}_e/\partial r_e)\mathbf{u}_e$, evaluated analytically.

Curved Beam Element (IGA Timoshenko)

When `--curved` is enabled, a cubic Bézier curve is fitted to each skeleton edge and a curved finite element is built on the same basis.

Bézier fitting: With endpoints $\mathbf{P}_0$ and $\mathbf{P}_3$ fixed, interior control points $\mathbf{P}_1$ and $\mathbf{P}_2$ are found by least-squares fitting. Two constraints prevent degenerate curves:

- **Chord monotonicity**: $0 \leq t_1 \leq t_2 \leq L$
- **Perpendicular bulge limit**: lateral deviation is clamped

Timoshenko formulation: A Timoshenko formulation captures axial-bending coupling from centreline curvature. The cubic Bernstein basis describes both geometry and displacement (isogeometric analysis). Four control points give 24 DOFs.

Static condensation: Interior DOFs are eliminated:

$$\mathbf{K}_{\text{cond}} = \mathbf{K}_{bb} - \mathbf{K}_{bi}\,\mathbf{K}_{ii}^{-1}\,\mathbf{K}_{ib} \tag{3.11}$$

where b and i denote boundary ($\mathbf{P}_0, \mathbf{P}_3$) and interior ($\mathbf{P}_1, \mathbf{P}_2$) DOFs, making the curved element a drop-in replacement.

3.3.6 Frame Re-optimisation (Stages 2–3)

Optimality Criteria Sizing (Stage 2)

`optimize_size()` adjusts every beam radius to minimise compliance while keeping total frame volume equal to the solid volume from Stage 1. In hybrid mode the constraint includes shell contributions: $\sum_i \pi r_i^2 L_i + \sum_j A_j h_j = V_{\text{target}}$.

$$r_i^{\text{new}} = \text{clip}_{[r_i(1-m),\, r_i(1+m)]}\left(r_i \left(\frac{-\partial C/\partial r_i}{\lambda\, \partial V/\partial r_i}\right)^{\eta}\right) \tag{3.12}$$

where $m = 0.2$ limits change per iteration, $\eta = 0.5$ is a damping exponent, and λ enforces the volume constraint via bisection. Both sensitivities ($\partial C/\partial r_i$, $\partial V/\partial r_i = 2\pi r_i L_i$) are evaluated analytically.

Convergence: Iteration terminates when the maximum radius change falls below 10^{-3}, or after 50 iterations.

L-BFGS-B Layout Optimisation (Stage 3)

`optimize_layout()` repositions the free nodes:

$$\min_{\mathbf{x}}\; C(\mathbf{x}) \quad \text{s.t. } \mathbf{x}_{\min} \leq \mathbf{x} \leq \mathbf{x}_{\max} \tag{3.13}$$

Each free node may move a limited distance. BC-tagged and loaded nodes remain fixed. After convergence, nodes within `snap_dist` are merged.

Alternating Convergence

Size and layout alternate in cycles (Stage 2 then Stage 3), controlled by `--opt_loops` (default 2). Size runs first because uniform radii give all members near-identical sensitivities; once a non-uniform distribution is established, layout can meaningfully reposition nodes.

3.3.7 External Mesh Input

External TO results from commercial solvers can be imported as STL/OBJ meshes via `mesh_voxelizer`. The mesh is voxelised at `--mesh_pitch` using trimesh, with interior automatically filled. EDT-based radius assignment is used.

3.3.8 Symmetry Enforcement

`src.optimization.symmetry` detects symmetric node/edge pairs by reflecting positions across user-specified planes, then enforces symmetry via radius averaging (size opt) and a soft penalty plus exact projection (layout opt).

3.4 JSON Output Schema

Every stage of the pipeline writes or reads a `.json` file.

3.4.1 Top-Level Structure

```
{
  "metadata": { ... },
  "graph":    { ... },
  "curves":   [ ... ],
  "plates":   [ ... ],
  "joints":   [ ... ],
  "history":  [ ... ]
}
```

3.4.2 `metadata`

Table 3.2: Metadata fields.

Key	Type	Description
`method`	str	`"Baseline Yin"`
`pitch`	float	Voxel size in mm
`units`	str	`"mm"`
`target_volume`	float	Total solid volume (mm^3) from Stage 0
`design_bounds`	[[x,y,z],[x,y,z]]	Axis-aligned bounding box
`vol_thresh`	float	Density threshold used to binarise
`plate_mode`	str	`"bspline"`, `"voxel"`, or `"mesh"`
`load_force`	[fx, fy, fz]	Applied load vector

3.4.3 graph

Table 3.3: Graph fields.

Key	Type	Description
`nodes`	`[[x,y,z], ...]`	Node world positions (mm), length N
`edges`	`[[u,v,r], ...]`	u, v = 0-based node indices; r = radius (mm)
`node_tags`	`{"i": tag, ...}`	1 = fixed, 2 = loaded, 0 = free

3.4.4 curves

One entry per edge, in the same order as `graph.edges`.

Straight beam (no `--curved`):

```
{ "points": [[x, y, z, r], ...] }
```

Curved beam (`--curved`):

```
{
  "ctrl_pts": [[x1,y1,z1], [x2,y2,z2]],
  "points":   [[x, y, z, r], ...],
  "radius":   r
}
```

3.4.5 plates

Each plate entry:

```
{
  "id":                    0,
  "voxels":                [[x,y,z], ...],
  "vertices":              [[x,y,z], ...],
  "faces":                 [[i,j,k], ...],
  "thickness":             [t0, t1, ...],
  "connection_node_ids":   [n0, n1, ...],
  "bspline_surface":       { "grid_u": [...], "grid_v": [...], "points": [...] }
}
```

3.4.6 joints

```
{
  "plate_id":  0,
  "node_id":   3,
  "location":  [x, y, z],
  "direction": [dx, dy, dz],
  "radius":    r,
  "type":      "fillet"
```

```
}
```

3.4.7 `history`

A list of snapshots, one per processing step. Two types:

Voxel snapshot (initial solid, skeleton, zone classification):

```
{
  "type":   "voxels",
  "step":   "1_Initial_Voxels",
  "points": [[x,y,z], ...],
  "colors": [[r,g,b], ...]
}
```

Graph snapshot (after each postprocessing step):

```
{
  "type":   "graph",
  "step":   "3_Raw_Graph",
  "nodes":  [[x,y,z], ...],
  "edges":  [[u, v, r, [waypoints...]], ...],
  "plates": [ ... ]
}
```

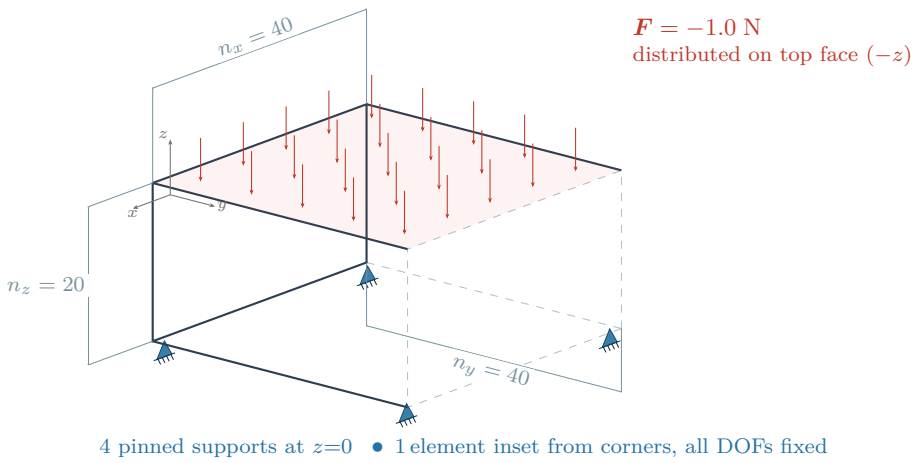

Figure 3.2: Roof structure problem setup ($40 \times 40 \times 20$ elements).

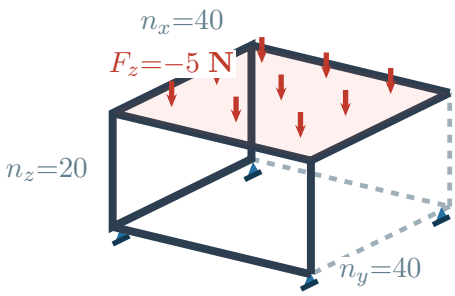

Figure 3.3: Frame-supported beam problem setup.

Chapter 4

CLI Reference

4.1 `run_pipeline.py` — Main Pipeline

`run_pipeline.py` is the primary entry point. It orchestrates all four pipeline stages and writes the history JSON consumed by the FreeCAD macro.

4.1.1 Synopsis

```
python run_pipeline.py [OPTIONS]
```

4.1.2 Examples

Full beam-only run (Stages 0–3):

```
python run_pipeline.py \
    --nelx 60 --nely 20 --nelz 4 \
    --volfrac 0.3 \
    --opt_loops 2 \
    --output cantilever.json
```

Skip Stage 0 (reuse existing NPZ):

```
python run_pipeline.py \
    --skip_top3d \
    --top3d_npz output/hybrid_v2/my_top3d.npz \
    --output my_result.json
```

Hybrid beam+plate mode:

```
python run_pipeline.py \
    --skip_top3d \
    --top3d_npz output/hybrid_v2/roof_top3d.npz \
    --hybrid \
```

```
    --min_plate_size 8 --flatness_ratio 7 \
    --output roof_hybrid.json
```

4.1.3 Argument Reference

Table 4.1: Complete argument reference for `run_pipeline.py`.

Argument	Default	Description
Top3D Design Domain		
`--nelx`	60	Elements in X (domain length)
`--nely`	20	Elements in Y (domain height)
`--nelz`	4	Elements in Z (domain depth)
`--volfrac`	0.3	Target volume fraction (0–1)
`--penal`	3.0	SIMP penalisation exponent
`--rmin`	1.5	Density filter radius (elements)
`--max_loop`	50	Max Top3D iterations
Load Definition		
`--load_x`	`nelx`	Load node X index
`--load_y`	`nely`	Load node Y index
`--load_z`	`nelz/2`	Load node Z index
`--load_fx`	—	Force X component (N)
`--load_fy`	—	Force Y component (N)
`--load_fz`	—	Force Z component (N)
`--load_dist`	`point`	Distribution: `point`, `surface_top`, `surface_bottom`
Skeletonisation		
`--pitch`	1.0	Voxel size in mm
`--max_iters`	50	Max thinning iterations
`--vol_thresh`	0.3	Density threshold for binarisation
`--prune_len`	2.0	Prune branch tips shorter than X mm
`--collapse_thresh`	2.0	Collapse edges shorter than X mm
`--rdp`	1.0	RDP polyline simplification epsilon (0 = off)
`--radius_mode`	`uniform`	`uniform` = volume-matching; `edt` = geometric
Hybrid Beam+Plate Mode		

Continued on next page

Argument	Default	Description
`--hybrid`	off	Enable hybrid pipeline
`--detect_plates`	`auto`	`auto`, `off`, or `force`
`--plate_mode`	`bspline`	`bspline`, `voxel`, or `mesh`
`--plate_thickness_ratio`	0.15	Max plate half-thickness as fraction of diagonal
`--min_plate_size`	4	Min skeleton voxels for plate classification
`--flatness_ratio`	3.0	PCA eigenvalue ratio threshold
`--junction_thresh`	4	Min neighbours for junction/plate classification
`--min_avg_neighbors`	3.0	Min average neighbour count
Optimisation		
`--optimize`	off	Enable Stages 2 & 3
`--opt_loops`	2	Size + Layout iteration loops
`--iters`	50	Max iterations per stage
`--limit`	5.0	Move limit for layout (mm)
`--snap`	5.0	Node snap distance (mm)
`--prune_opt_thresh`	0.0	Post-opt pruning threshold
`--problem`	`tagged`	BC problem config
Curved Beams		
`--curved`	off	Fit cubic Bézier curves; enables IGA elements
External Mesh Input		
`--mesh_input`	—	Path to STL/OBJ mesh (skips Top3D)
`--mesh_pitch`	1.0	Voxel pitch for mesh voxelisation (mm)
Symmetry		
`--symmetry`	—	Symmetry axis: `x`, `y`, or `z`
`--sym_tol`	1.0	Symmetry plane snap tolerance (mm)
Output & Control		
`--output`	`full_control_beam.json`	Output filename
`--output_dir`	`output/hybrid_v2`	Output directory

Continued on next page

Argument	Default	Description
`--visualize`	off	Open interactive Open3D windows
`--skip_top3d`	off	Skip Stage 0; use `--top3d_npz`
`--top3d_npz`	—	Path to existing `.npz`

4.2 `run_top3d.py` — Topology Optimisation

Standalone script that runs Python Top3D and writes a `.npz` density field.

4.2.1 Synopsis and Examples

```
python run_top3d.py [OPTIONS]
```

Short cantilever (fast):

```
python run_top3d.py \
    --nelx 60 --nely 20 --nelz 4 \
    --volfrac 0.3 --penal 3.0 --rmin 1.5 \
    --problem cantilever \
    --output output/hybrid_v2/cantilever_top3d.npz
```

Roof structure:

```
python run_top3d.py \
    --nelx 40 --nely 40 --nelz 20 \
    --volfrac 0.05 --penal 3.0 --rmin 1.5 --max_loop 100 \
    --load_fz -100.0 \
    --problem roof \
    --output output/hybrid_v2/roof_top3d.npz
```

4.2.2 Argument Reference

4.2.3 Problem Types

4.3 `tune_parameters.py` — Parameter Tuning

Uses Optuna to search for the best skeletonisation parameters for a given NPZ.

4.3.1 Synopsis and Examples

```
python tune_parameters.py NPZ_PATH [OPTIONS]
```

100-trial search:

```
python tune_parameters.py \
```

Table 4.2: Arguments for `run_top3d.py`.

Argument	Default	Description
`--nelx`	60	Elements in X
`--nely`	20	Elements in Y
`--nelz`	4	Elements in Z
`--volfrac`	0.3	Target volume fraction
`--penal`	3.0	SIMP penalisation exponent
`--rmin`	1.5	Filter radius (elements)
`--max_loop`	50	Max solver iterations
`--load_x`	`nelx`	Load node X index
`--load_y`	`nely`	Load node Y index
`--load_z`	`nelz/2`	Load node Z index
`--load_fx`	0.0	Force X (N)
`--load_fy`	-1.0	Force Y (N)
`--load_fz`	0.0	Force Z (N)
`--problem`	`cantilever`	Problem type
`--load_dist`	`point`	Distribution mode
`--output`	`python_top3d_result.npz`	Output path

Table 4.3: Available problem types.

Problem	Boundary conditions
`cantilever`	All DOFs fixed at $x = 0$; point load at (x, y, z)
`roof`	4 corner columns ($z = 0$) fixed; central downward load
`bridge`	Bottom surface ($z = 0$) fixed; distributed load at top
`deck`	Same as bridge but with symmetric load points

```
    output/hybrid_v2/matlab_replicated_top3d.npz \
    --trials 100
```

4.3.2 Argument Reference

Table 4.4: Arguments for `tune_parameters.py`.

Argument	Default	Description
`npz_path` (positional)	—	Path to Top3D `.npz` density field
`--trials`	100	Number of Optuna trials
`--output_dir`	`output/tuning`	Directory for trial outputs

4.3.3 Search Space

Table 4.5: Optuna search space.

Parameter	Range	Maps to
`prune_len`	0.5–5.0 (float)	`--prune_len`
`collapse_thresh`	1.0–5.0 (float)	`--collapse_thresh`
`rdp`	0.5–5.0 (float)	`--rdp`
`radius_mode`	`uniform` / `edt`	`--radius_mode`

4.3.4 Objective Function

Minimises a composite geometry fidelity score:

```
score = volume_error + 0.1 * complexity_penalty + 0.2 * coverage_penalty
```

where:

- `volume_error` — relative error between frame volume and target
- `complexity_penalty` — normalised edge count
- `coverage_penalty` — skeleton voxel coverage fraction

Chapter 5

API Reference

This chapter documents the public API of every major module in the `src/` package tree. All public functions use Google-style docstrings.

5.1 Pipelines — `src.pipelines.baseline_yin`

5.1.1 `reconstruct`

Main Stage 1 orchestrator. Loads a Top3D `.npz`, runs thinning and graph extraction, and exports the result to `.json`.

`reconstruct_npz(npz_path, output_json, **kwargs)` *src.pipelines.baseline_yin.reconstruct*

Converts a Top3D voxel density field (`.npz`) into a beam/plate graph for downstream FEM optimisation.

Args:

`npz_path`	Path to the Top3D `.npz` file.
`output_json`	Output `.json` file path.
`**kwargs`	Pipeline options (`pitch`, `vol_thresh`, `prune_len`, `collapse_thresh`, `rdp`, `radius_mode`, `hybrid`, etc.).

Returns: Dict containing the reconstructed graph and metadata.

`export_to_json(nodes_dict, edges, output_path, ...)` *src.pipelines.baseline_yin.reconstruct*

Serialises the skeleton graph and plate data to a `.json` file.

Args:

`nodes_dict`	Dict mapping node index to (x, y, z) position.
`edges`	List of edge tuples `[u, v, weight, pts, r]`.

`output_path`	Destination `.json` path.

5.1.2 `thinning`

Yin 3-D parallel medial-axis thinning (Algorithm 3.1 from Yin 2002).

`thin_grid_yin(volume, tags, max_iters, record_iterations, mode, ...)`
src.pipelines.baseline_yin.thinning

Reduces a binary volume to its topological skeleton while preserving connectivity.

Args:

`volume`	3-D boolean array (the binary solid).
`tags`	3-D int array of BC tags (tag > 0 = protected).
`max_iters`	Maximum number of thinning iterations.
`mode`	Thinning mode: 0 (curve), 1 (surface), 3 (hybrid).

Returns: Thinned boolean volume (same shape as input).

5.1.3 `topology`

Low-level topological predicates accelerated with Numba. Key function: `is_simple_point(neighbourhood)` — returns `True` if removing the centre voxel preserves topology.

5.1.4 `graph`

Skeleton → graph extraction and zone classification.

`extract_graph(skeleton, pitch, origin, tags, hybrid_mode, ...)`
src.pipelines.baseline_yin.graph

Converts a medial-axis skeleton into a node/edge graph.

Args:

`skeleton`	3-D boolean skeleton array.
`pitch`	Voxel pitch in mm.
`origin`	World-coordinate origin `(x, y, z)`.
`tags`	BC tag array.
`hybrid_mode`	If `True`, extract plate zones.

Returns: Tuple of `(nodes_dict, edges, vertex_types, node_tags)`.

`classify_skeleton_post_thinning(skeleton, min_plate_size, flatness_ratio, ...)`
src.pipelines.baseline_yin.graph

Two-signal zone classification for hybrid beam+plate mode. Combines set-difference (Signal A: mode 3 \ mode 0) with octant plane patterns (Signal B) to partition the skeleton into plate and beam voxels.

Args:

`skeleton`	3-D boolean skeleton (mode 3).
`min_plate_size`	Minimum voxel count for a plate region.
`flatness_ratio`	PCA eigenvalue ratio threshold.

Returns: 3-D int array: 1 = plate, 2 = beam, 0 = background.

5.1.5 `postprocessing`

Graph refinement: pruning, edge collapse, radius computation.

`collapse_short_edges(nodes, edges, threshold, node_tags)`
src.pipelines.baseline_yin.postprocessing

Merges node pairs connected by edges shorter than `threshold`. BC-tagged nodes anchor the merge.

`prune_branches(nodes, edges, min_len, node_tags)` *src.pipelines.baseline_yin.postprocessing*

Iteratively removes degree-1 leaf nodes on edges shorter than `min_len`. BC-tagged endpoints are always retained.

`simplify_graph_geometry(nodes, edges, epsilon)` *src.pipelines.baseline_yin.postprocessing*

Ramer–Douglas–Peucker simplification of edge waypoints.

`compute_uniform_radii(nodes, edges, total_voxel_volume, pitch)`
src.pipelines.baseline_yin.postprocessing

Assigns uniform radius: $r_0 = \sqrt{V_{\text{solid}}/(\pi \sum L_i)}$.

`compute_edge_radii(nodes, edges, edt_volume, pitch, origin)`
src.pipelines.baseline_yin.postprocessing

Assigns per-edge radius from the median EDT value along each edge.

5.1.6 `plate_extraction`

Plate region recovery for hybrid beam+plate mode.

`extract_plates_v2(plate_skeleton, plate_labels, solid, edt, pitch, origin, ...)`
src.pipelines.baseline_yin.plate_extraction

Extracts plate geometry from classified skeleton zones. Produces solid mesh and mid-surface with per-vertex thickness.

`recover_plate_regions_from_skeleton(skeleton_zone_mask, plate_labels, original_solid, edt)` *src.pipelines.baseline_yin.plate_extraction*

Recovers full-thickness plate regions by morphological dilation into the original solid, constrained to avoid beam zones.

5.1.7 `joint_creation`

Beam-plate interface joint snapping.

`create_beam_plate_joints(nodes_dict, edges, node_tags, plates_data, ...)`
src.pipelines.baseline_yin.joint_creation

Snaps beam endpoints near plate boundaries onto shared nodes using KD-tree lookup. Creates bridging edges for isolated beam endpoints.

5.1.8 `surface_fitting`

B-spline plate mid-surface fitting via least-squares.

5.1.9 `visualization`

Open3D and Matplotlib visualisation utilities. Requires `open3d`. Key functions: `viz_voxels_density()`, `show_density_colorbar()`.

5.1.10 `mesh_voxelizer`

External STL/OBJ mesh voxelisation for the `--mesh_input` path. Uses trimesh to voxelise at user-specified pitch, with interior fill.

5.2 Optimization — `src.optimization`

5.2.1 `top3d`

Pure-Python 3-D topology optimiser (SIMP + OC + mesh-independence filter).

`class Top3D(nelx, nely, nelz, volfrac, penal=3.0, rmin=1.5)` *src.optimization.top3d*

SIMP topology optimiser.

Args:

`nelx, nely, nelz`	Domain dimensions (elements).
`volfrac`	Target volume fraction.
`penal`	Penalisation exponent.
`rmin`	Filter radius.

`Top3D.optimize(max_loop=50, tolx=0.01)` *src.optimization.top3d*

Runs the SIMP optimisation loop.

Args:

`max_loop`	Maximum number of iterations.
`tolx`	Convergence tolerance on max density change.

Returns: `xPhys` — optimised density field, shape `(nely, nelx, nelz)`.

`Top3D.set_load(load_dof, magnitude)` *src.optimization.top3d*

Sets a concentrated load at the specified DOF index.

`Top3D.set_fixed_dofs(fixed)` *src.optimization.top3d*

Sets the list of constrained DOF indices.

`Top3D.set_passive_void(passive)` *src.optimization.top3d*

Marks voxels as passive void (excluded from design domain).

`lk_H8(nu=0.3)` *src.optimization.top3d*

Returns the 24×24 element stiffness matrix for an 8-node hexahedral element with Poisson's ratio `nu`.

5.2.2 `fem`

3-D frame FEA with straight Euler–Bernoulli and curved IGA Timoshenko beam elements.

`solve_frame(nodes, edges, radii, E, nu, loads, bcs, plates=None, plate_thicknesses=None)` *src.optimization.fem*

Assembles and solves a 3-D frame system with straight beam elements.

Args:

`nodes`	$(N, 3)$ array of node positions.
`edges`	$(M, 2)$ array of edge connectivity.
`radii`	$(M,)$ array of beam radii.

`E`	Young's modulus.
`nu`	Poisson's ratio.
`loads`	Dict of node index → force vector.
`bcs`	Dict of node index → constrained DOFs.

Returns: Tuple of `(u, compliance, elements, ...)`.

`compute_frame_gradients(nodes, edges, radii, u_total, E, nu)` *src.optimization.fem*

Computes compliance sensitivities $\partial C/\partial r_e$ for each beam.

Returns: $(M,)$ array of gradients.

`compute_element_stiffness(E, A, Iy, Iz, J, G, L)` *src.optimization.fem*

Returns the 12×12 local stiffness matrix for a straight beam element.

`rotation_matrix(vec, roll=0.0)` *src.optimization.fem*

Returns the 12×12 rotation matrix from local beam axis to global frame.

`solve_curved_frame(...)` *src.optimization.fem*

Mixed straight/curved assembly. Elements without control points use Euler–Bernoulli; those with control points use IGA Timoshenko (condensed).

5.2.3 `size_opt`

Optimality Criteria beam radius (cross-section) optimisation.

`optimize_size(nodes, edges, initial_radii, problem, E, vol_fraction, ...)`
src.optimization.size_opt

OC-based radius optimisation. Iterates until maximum radius change $< 10^{-3}$ or `max_iters` is reached.

Returns: Tuple of `(radii, c_initial, c_final, history)`.

5.2.4 `layout_opt`

L-BFGS-B node-position layout optimisation for beam frames.

`optimize_layout(nodes, edges, radii, problem, E, move_limit, ...)`
src.optimization.layout_opt

Moves free node positions along compliance gradients. Fixed and loaded nodes are frozen throughout.

Returns: Tuple of `(nodes_opt, edges_opt, radii_opt, tags_opt, c_initial, c_final)`.

5.2.5 `symmetry`

Mirror-half skeleton symmetry enforcement for layout optimisation. Detects symmetric node/edge pairs, enforces radius averaging, and constrains nodes on the symmetry plane.

5.3 Problems — `src.problems`

Problem configuration classes define boundary conditions. All classes expose an `apply(nodes)` method that returns `(loads, bcs)` dictionaries.

5.3.1 Factory: `load_problem_config`

`load_problem_config(problem_name)` *src.problems*

Returns the appropriate problem class for the given name string (`"tagged"`, `"cantilever"`, `"rocker_arm"`, etc.).

5.3.2 `TaggedProblem`

Tag-based BC assignment from node tags embedded in the pipeline JSON. This is the default problem type (`--problem tagged`). Reads `node_tags` from the JSON and applies fixed DOFs / loads accordingly.

5.3.3 `CantileverSetup`

Standard cantilever: fixed at $x = 0$, point load at $x = n_x$.

5.3.4 `GenericProblem`

Loads BC configuration from an external JSON file (`pipeline_bcs.json`).

5.4 Curves — `src.curves.spline`

Cubic Bézier curve utilities for smooth beam geometry. Cubic form: $P(t) = (1-t)^3 P_0 + 3t(1-t)^2 P_1 + 3t^2(1-t)P_2 + t^3 P_3$, $t \in [0, 1]$.

`fit_cubic_bezier(p0, p3, interior_pts)` *src.curves.spline*

Least-squares fitting of interior control points P_1, P_2 through skeleton waypoints, parameterised by cumulative arc-length fraction.

`sanitize_bezier_ctrl_pts(p0, p3, p1, p2, max_bulge_ratio=0.2)` *src.curves.spline*

Enforces chord monotonicity ($0 \leq t_1 \leq t_2 \leq L$) and perpendicular bulge limit ($\|P_{1\perp}\|, \|P_{2\perp}\| \leq 0.2L$) to prevent self-intersecting curves.

`sample_bezier(p0, p1, p2, p3, N)` *src.curves.spline*

Uniform parameter sampling of a cubic Bézier curve.

`bezier_arc_length(p0, p1, p2, p3, n_quad)` *src.curves.spline*

Gauss–Legendre quadrature for arc length.

5.5 Export — `src.export.freecad_reconstruct`

FreeCAD macro for importing pipeline JSON into a FreeCAD document. Creates swept beam tubes, Bézier curves, plate polygon faces, fused solid body, history timeline groups, and density legend cubes.

This module runs inside the FreeCAD Python interpreter and cannot be imported in a standard Python environment.

5.6 Tuning — `src.tuning`

5.6.1 `pipeline_runner`

Subprocess wrapper that runs a single reconstruction trial with a given parameter set and returns the output JSON path.

5.6.2 `metrics`

Geometry fidelity metric extraction from reconstruction output. Computes volume error, complexity penalty, and coverage penalty for the Optuna objective function.

Chapter 6

Developer Guide

6.1 Contributing

6.1.1 Branch Policy

All active development happens on **Main_V2**. Documentation is built from this branch only. Do not open PRs against legacy branches.

6.1.2 Code Style

- PEP 8 formatting (`black` recommended, line length 100).
- Google-style docstrings for all public functions:

```python
def foo(x: int) -> float:
    """One-line summary.

    Args:
        x: Description of x.

    Returns:
        Description of return value.
    """
```

Numba-decorated functions (`@njit`) are exempt from docstring requirements but should have a plain comment block.

6.1.3 Adding a New Problem

1. Create `src/problems/my_problem.py` with a class `MyProblem`.
2. Implement `apply(nodes) -> tuple[dict, dict]` returning `(loads, bcs)`.
3. Register it in `load_problem_config()`.
4. Add an `elif problem_name == 'my_problem':` branch in `run_top3d.py`.

6.1.4 Adding a New Thinning Mode

1. Add a new `elif args.mode == N:` branch inside `thin_grid_yin`.
2. Define the candidate-selection and deletion logic.
3. Add tests in `tests/test_thinning.py`.

6.1.5 Building the Docs

```
cd docs
pip install sphinx sphinx-rtd-theme
make clean html
open _build/html/index.html
```

6.2 Testing and Validation

The project does not yet have a formal test suite. The following manual validation commands serve as regression checks.

6.2.1 Smoke Test — Import Check

```
python -c "
from src.optimization.top3d import Top3D
from src.pipelines.baseline_yin.reconstruct import reconstruct_npz
from src.optimization.size_opt import optimize_size
from src.optimization.layout_opt import optimize_layout
from src.problems import load_problem_config
print('All imports OK')
"
```

6.2.2 Beam-Only Regression

```
python run_pipeline.py \
    --skip_top3d \
    --top3d_npz output/hybrid_v2/matlab_replicated_top3d.npz \
    --output regression_beam.json \
    --opt_loops 1
```

Expected: pipeline prints `PIPELINE COMPLETE` with no tracebacks.

6.2.3 Hybrid Regression

```
python run_pipeline.py \
    --skip_top3d \
    --top3d_npz output/hybrid_v2/matlab_replicated_top3d.npz \
    --hybrid \
    --output regression_hybrid.json
```

Expected: `PIPELINE COMPLETE (Unoptimized Hybrid)` with plate count ≥ 1.

6.2.4 Parameter Tuning Validation

```
python tune_parameters.py \
    output/hybrid_v2/matlab_replicated_top3d.npz \
    --trials 10 \
    --output_dir output/tuning_test
```

6.2.5 Docs Build Check

```
cd docs && make clean html 2>&1 | grep -E "(WARNING|ERROR|error)"
```

A clean build should report zero errors and minimal warnings.